

COMMUNITY LIBRARY MANAGEMENT SYSTEM

REQUIREMENTS UNDERSTANDING

NAME: KEERTHIKESWARAN

System Overview

The primary objective of the application is to manage the daily operations of a small community library through a structured console-based interface. The system focuses on three core pillars: user accountability, asset management, and financial integrity. It relies on a three-tier architecture to separate data access, business logic, and user interaction, utilizing PostgreSQL and Entity Framework Core for persistence.

Member Management

Member operations involve complete lifecycle management, from registration to deactivation. Every member is assigned one of three membership tiers: Basic, Student, or Premium. Searching for members must be flexible, allowing lookups via email or phone number. Membership status whether Active or Inactive is the fundamental gateway for all lending activities.

Book and Inventory Management

The system distinguishes between a book title and its physical copies. While a book record holds metadata like title, author, and category, the book copies track the physical state and availability of each unit. This allows for detailed tracking of inventory, including marking specific copies as damaged or unavailable without affecting the entire title record. Automated category mapping ensures that books are always categorized correctly, even when new categories are introduced during book addition.

Borrowing Rules and Business Logic

Lending is governed by strict, tier-based constraints. The system enforces different rules based on the membership type:

Basic Members: Allowed 2 active borrowings for up to 7 days.

Student Members: Allowed 3 active borrowings for up to 10 days.

Premium Members: Allowed 5 active borrowings for up to 15 days.

Borrowing is strictly prohibited if the member is inactive, has reached their borrowing limit, or possesses unpaid fines exceeding ₹500. Furthermore, duplicate active borrowings of the same book title by the same member are blocked to ensure fair distribution of resources.

Return Workflow and Fine Calculation

The return process is a multi-step transaction designed to ensure the physical and digital states of the library are in sync. When a return is initiated, the system calculates late fees at a standard rate of ₹10 per delayed day. The status of the book copy transitions through an intermediate state during return processing to ensure that inventory is not marked as available until it has been inspected. Once a return is finalized, the borrowing record is closed, and the return date is permanently recorded.

Fine Management

Financial oversight is maintained through a dedicated fine tracking module. Members can view their pending fines, payment history, and clear their dues. The system uses these financial records as a hard block during the borrowing process if the total unpaid amount crosses the ₹500 threshold. Every fine record is associated with a specific borrowing transaction for full auditability.

Database Persistence and Transactions

Data integrity is maintained through the use of PostgreSQL and EF Core. A critical requirement is the use of database transactions during the borrowing process. This ensures that the validation of the member, the check for unpaid fines, the verification of copy availability, and the creation of the borrowing record all succeed or fail as a single unit. Additionally, PostgreSQL functions are utilized for complex data retrieval, such as calculating total unpaid fines or generating borrowing summaries, which are then called directly from the application code.

Reporting and Analytics

- The system provides a comprehensive suite of reports to assist administrators in library oversight. These include:
- Lists of currently borrowed and overdue books.
- Identification of members with outstanding financial dues.
- Analytics on the most borrowed books and available books filtered by category.
- Detailed borrowing histories for individual members.
- Planned Additional Features
- The following enhancements are incorporated to provide a more professional and robust experience:

Detailed Fine Remarks: Fines are automatically tagged with descriptive remarks such as "Late Return," "Damaged Book," or "Late Return and Damaged Book" to provide clarity for members.

Case-Insensitive Search: Search functionality for books and members is designed to be case-insensitive to improve the user experience and reduce search failures.

Additional Features Planned

Administrative Return Approval: Instead of an automatic return, the system implements a two-stage workflow where a member submits a return, and an administrator must verify the condition (Good/Damaged) before the transaction is finalized.

Customizable Fine Amount: The Admin can customize the fine amount which has been stored in the FineConfiguration Table.

Isolated Fine Configuration and Fine Amount: The system intentionally separates active business rules (FineConfiguration) from transactional history (FineCalculation) without establishing a direct Foreign Key relationship. When a return is processed, the system computes the exact monetary penalty using the current active rates and saves that absolute value into the member's fine record. This architectural decision prevents the recalculation trap, ensuring that if an Administrator updates fine rates in the future, past financial debts remain permanently frozen and accurate, protecting the historical integrity of the library's records.

Decoupled Password Table: By decoupling the authentication data into a dedicated Passwords table using universal identity prefixes (e.g., mem_1 for members, adm_1 for admins), the system avoids duplicating logic across multiple tables. During login, the system looks up the user, safely hashes their inputted password, and validates it against the stored hash, significantly enhancing security while keeping the data layer clean and focused purely on storage.

Automated Category Creation: During book addition, the system checks if the specified category exists. If not, it automatically creates a new category record with a unique ID, streamlining the cataloging process.

Edge Case Handled

=> The system is designed to handle several complex scenarios that often lead to data inconsistency:

Concurrent Return Prevention: The logic prevents a member from submitting multiple return requests for the same active borrowing, which could lead to duplicate fine calculations.

Damaged Copy Handling: If a book title has copies remaining but they are all marked as "Damaged," the system handles this nuance by either blocking the borrow or requiring an explicit confirmation from the member.

Nullable Type Safety: Extensive checks are implemented for nullable database fields (such as fine amounts or return dates) to prevent runtime crashes during mathematical operations or date comparisons.

Check Constraint Compliance: The application code is strictly aligned with database-level check constraints on statuses like copy status and return status, ensuring that the application never attempts to save an invalid state.

Empty Search Results: The search interface identifies when a query yields no results and provides immediate feedback, skipping unnecessary prompts for book IDs or borrowing details.